

Backend Engineering Challenge:

Distributed Video Processing Pipeline

Overview

We are developing an internal tool to assist our Data and Marketing teams with video insights. This challenge involves building a video processing pipeline consisting of two microservices. The goal is to extract frames from a video file at a specific rate and perform automated "face detection" on those frames.

Architecture

A. VideoAnalyzer Service

Responsible for job ingestion and frame extraction.

- **API Endpoint:** `POST /analyze`
- **Input:** A JSON payload containing a `file_path` (local path) and `fps` (requested frames per second).
- **Validation:** Ensure the `fps` parameter is strictly **2** or **4**.
- **Logic:**
 - **Identification:** Generate a unique ID for the video (you may use the filename for simplicity).
 - **Extraction:** Read the video file and extract frames based on the requested `fps`. *(e.g., if the source is 30fps and the user asks for 2fps, you should process every 15th frame).*
 - **Dispatch:** Send each extracted frame to the **StreamDetector** service.
- **Response:** The service should return a `200 OK` status code only after it has finished reading and dispatching all relevant frames.

B. StreamDetector Service

Acts as the computer vision engine.

- **Input:** Receives the frame data, `video_id`, and `frame_index`.
 - **Face Detection:** Utilize the provided mock function to "detect" faces. This function simulates a heavy Machine Learning workload.
 - **Output:** Construct a `RespObject` containing the `video_id`, `frame_number`, and the detected faces (bounding boxes).
 - **Final Action:** You do not need to implement the final network call to an external service. Simply prepare the final object and pass it to the provided `send_results_next_service` placeholder.
-

Technical Requirements

Scaling & Bottlenecks

- **Performance:** The system should strive for real-time video analysis capabilities.
- **Extensibility:** You are encouraged to use auxiliary services (e.g., **Message Brokers** like RabbitMQ/Kafka/Redis or **Databases**) to manage state, decoupling, and backpressure.
- Assume that both services can run on different machines.

Infrastructure

- Provide a `Dockerfile` for each service.
- Provide a `docker-compose.yml` that orchestrates the entire environment, including any auxiliary services used.

Language & Quality

- **Language:** The solution must be implemented using **Python 3.x**.
 - **Code Quality:** Adhere to Python best practices (PEP 8, type hinting, and intuitive variable naming). The structure should reflect senior-level cleanliness and maintainability.
 - **Robustness:** Ensure thorough input validation and error handling for the `/analyze` endpoint.
-

Provided Boilerplate

A boilerplate structure is provided in the attached `.zip` file. Please integrate your logic into the following components:

- **Detection Logic:** Located in `stream_detector/detector.py`.
 - **Response Handling:** Located in `stream_detector/detector_response_handling.py`.
 - **Videos directory:** a videos directory with an example video is provided.
-

Stage 2: Production Scaling (Interview Discussion)

Following the internal success of this tool, we intend to move it to a high-scale production environment. **During the follow-up interview, please be prepared to discuss:**

- How would you restructure the architecture differently to analyze hundreds of videos concurrently?
 - What changes would you make to ensure high availability and fault tolerance?
 - **Advantage:** Prepare a high-level architectural diagram to illustrate your proposed production-ready design.
-

Submission

Please provide your solution via one of the following:

1. A link to a private/public GitHub repository.
2. A compressed `.zip` file containing the source code and instructions on how to run it.